

LECTURE 03

DOM I, *the live page.*

HTML becomes a tree. JavaScript gets handles to that tree. Interaction starts there.

document

selectors

nodes

traversal

events

BEFORE WE DOM

Three JS tools you need today.

REST

`...nums`

Collects extra function arguments into one array.

SPREAD

`...arr`

Expands an array or object into individual pieces.

ARRAY CHECK

`Array.isArray`

Use it because `typeof []` says object.

```
function sum(...nums) { return nums.reduce((a, b) => a + b, 0) }  
const combined = [...a, ...b]  
Array.isArray(combined)    // true
```

THE CORE IDEA

HTML is source. DOM is memory.

The Document Object Model is the browser's live, tree-structured representation of the page.

change the DOM, change what the user sees.

TREE RULES

A page is a family tree.

```
document
  body
    main
      h1
      button
    footer
```

- ✓ One root node starts the tree.
- ✓ Nodes can have many children.
- ✓ Nodes have zero or one parent.
- ✓ Siblings share the same parent.

selectors find nodes. traversal moves between relatives.

BROWSER DEVTTOOLS

Your DOM microscope.

OPEN THE LAB

- ✓ Right-click → Inspect
- ✓ Chrome Mac: Cmd Option J
- ✓ Chrome Win/Linux: Ctrl Shift J

USE IT FOR

- ✓ Inspecting elements
- ✓ Trying DOM queries
- ✓ Reading console output
- ✓ Debugging event handlers

Elements tab shows the tree. Console lets JS touch it.

THE GLOBAL HANDLE

document is the front door.

```
// available globally in the browser
document

// find a specific element
const title = document.querySelector("h1")

// then read or change it
title.innerText
title.style.color = "blue"
```

document connects JavaScript to the live DOM tree.

Most DOM code is: start at document, find an element, do something to it.

don't manipulate the page blind. grab a handle first.

SELECTORS

Find the node you mean.

```
document.getElementById("myId")           // one element
document.getElementsByClassName("card")    // HTMLCollection
document.getElementsByTagName("div")       // HTMLCollection

document.querySelector(".card")           // first CSS selector match
document.querySelectorAll(".card")        // NodeList of all matches
```

Use IDs when there is exactly one meaningful element.

Use query selectors when your mental model is already CSS.

ARRAY-ISH

Collections are not real arrays.

HTMLCollection and NodeList look like arrays:

- ✓ They are zero-indexed.
- ✓ They have a length.
- ✓ You can access items by index.

But they do not reliably have `map`, `filter`, or `reduce`.

```
const nodes =
  document.querySelectorAll("li")

// convert to a real array
Array.prototype.slice.call(nodes)
Array.from(nodes)
[...nodes]

// now array methods are fair game
[...nodes].map(li => li.innerText)
```

RELATIVES

Once you have a node, you can move.

```
element.children  
element.firstChild  
element.lastChild  
element.nextElementSibling  
element.previousElementSibling  
element.parentElement
```

USE TRAVERSAL WHEN

- ✓ You found one element and need a nearby one.
- ✓ A click target lives inside a larger component.
- ✓ Parent/child structure matters more than class names.

search gets you there. traversal moves around.

MUTATION

Change the tree, change the page.

```
// inline style
document.getElementById("p2")
  .style.color = "blue"

// visible text
element.innerText = "Saved!"

// HTML inside an element
element.innerHTML = "<b>Bold</b>"

// form input value
input.value
```

```
// remove a known child
node.removeChild(oldNode)

// remove when you only have the child
oldNode.parentNode
  .removeChild(oldNode)

// modern shortcut also exists
oldNode.remove()
```

innerHTML is powerful. treat user input like it is radioactive.

INTERACTIVITY

Listen now. Run later.

```
document
  .getElementById("myId")
  .addEventListener("click", function(event) {
    console.log("clicked!", event)
  })
```

An event listener is a callback attached to an element.

The browser calls it when the named event happens.

- ✓ click
- ✓ mouseover
- ✓ keyup / keydown
- ✓ scroll

EVENT OBJECT

The browser hands you the receipt.

WHAT HAPPENED?

`event.type`

WHO CAUSED IT?

`event.target`

KEYBOARD

`event.key`

MOUSE X/Y

`clientX / clientY`

WHEN?

`timeStamp`

CANCELED?

`defaultPrevented`

event.target is the one you'll inspect constantly.

THE LOOP

Search. Traverse. Change. Listen.

DOM I skill check

- ✓ Find elements with selectors
- ✓ Move to related nodes
- ✓ Read and write content
- ✓ React to user events

```
const button = document.querySelector("button")

button.addEventListener("click", (event) => {
  event.target.innerText = "Clicked"
})
```